

JavaScript 예외처리

학습 체크리스트 (1/2)

- ☐ 에러 방어를 위한 `try-catch` 문의 기본 구조 및 작동 흐름 이해
- ☐ `catch(error)` 매개변수를 통한 에러 객체(`message`, `name`, `stack`) 활용
- ☐ `instanceof` 연산자 기반의 에러 유형별 분기 감지 및 커스텀 익셉션 패턴 적용
- ☐ 예외 조건 발생 시 `throw` 키워드를 이용한 강제 에러 발생 및 제어

학습 체크리스트 (2/2)

- ☐ 상위 호출자로 에러 처리를 위임하는 `rethrow` 패턴의 이해 및 활용
- ☐ 예외 발생 여부와 무관하게 필수 실행되는 `finally` 블록의 Cleanup 원리 숙지
- ☐ `finally` 블록 내 `return` 선언 시 앞선 반환값을 덮어쓰는(Override) 동작 파악
- ☐ `try-catch-finally` 블록 레벨 스코프와 상위 `let` 공통 변수 설계 방식 파악

예외 처리 (try-catch)

예외 처리 (try-catch)

런타임 에러 발생 시 프로그램 즉각 종료를 방지하고 예외 대응용 대안 코드를 실행하는 제어 흐름 구조

- 에러 객체 매개변수: `catch(err)` 의 에러 객체는 `name`, `message`, `stack` 등의 속성 제공
- 매개변수 생략: ES2019부터 `catch` 매개변수가 불필요하다면 `catch { ... }` 형태로 생략 가능
- 커스텀 에러 상속: 내장 `Error` 클래스를 상속받아 커스텀 에러 클래스 정의 및 구분 처리 가능

커스텀 에러 클래스 선언

```
1  class ValidationError extends Error {  
2      constructor(message) {  
3          super(message);  
4          this.name = "ValidationError";  
5      }  
6  }  
7  
8  function verifyUser(user) {  
9      if (!user.name) {  
10         throw new ValidationError("이름 누락!");  
11     }  
12 }
```

예외 감지 및 분기 처리

```
1  try {
2    const user = {}; // 이름 누락 상황
3    verifyUser(user);
4  } catch (error) {
5    if (error instanceof ValidationError) {
6      console.error(`[검증 오류] ${error.message}`);
7    } else {
8      console.error(`[기타 오류] ${error.message}`);
9    }
10 }
```

에러 던지기 (throw)

에러 던지기 (throw)

특정 예외 조건 감지 시 예외 발생을 알리기 위해 강제로 에러를 생성하여 던지는 키워드

- **에러 객체 던지기**: 관례 및 스택 추적 디버깅 정보를 위해 문자열 등이 아닌 `new Error()` 등의 객체를 던지는 것을 권장
- **호출 스택 전파**: `throw` 실행 시 현재 흐름이 즉시 중단되고 호출 스택을 거슬러 올라가며 가장 가까운 `catch` 블록을 탐색
- **rethrow 패턴**: 감지한 에러 중 감당할 수 없거나 상위 위임이 필요한 경우 상위 호출자에게 다시 에러를 던져 예외 처리를 위임

예외 발생 및 위임(Rethrow)

```
1  function loadConfig(configJson) {  
2      try {  
3          const config = JSON.parse(configJson);  
4          if (!config.apiUrl) throw new SyntaxError("URI 누락!");  
5          return config;  
6      } catch (error) {  
7          if (error instanceof SyntaxError) {  
8              return { apiUrl: "https://default-api.com" }; // 기본값 복구  
9          }  
10         throw error; // 처리 불가능한 에러는 상위로 위임 (Rethrow)  
11     }  
12 }
```

최상위 예외 감지 및 처리

```
1  try {  
2    loadConfig("{ broken json ... }"); // JSON 파싱 실패 오류 발생  
3  } catch (err) {  
4    console.log("최상단 감지 에러:", err.message); // 상위 위임 확인  
5  }
```

마무리 블록 (finally)

마무리 블록 (finally)

에러 발생 여부 및 return 실행 여부와 상관없이 제어 흐름의 가장 마지막에 무조건 실행되는 정리용 블록

- **클린업 코드**: 파일 닫기, 네트워크 연결 종료, 로딩 UI 비활성화 등 필수 실행되어야 하는 뒷정리 코드를 배치함
- **return 흐름 제어**: `try` 나 `catch` 에서 `return` 이 먼저 실행되어도 함수의 반환 전에 `finally` 블록이 먼저 호출됨
- **return 덮어쓰기**: `finally` 내부에서 `return` 을 선언하면 이전 블록의 `return` 값은 덮어쓰워져(Override) 소멸함

finally 내 return 덮어쓰기

```
1 function checkReturnFlow() {  
2   try {  
3     console.log("try 블록 진입");  
4     return "try 결과"; // 즉시 반환하지 않고 finally로 분기  
5   } finally {  
6     console.log("finally 블록 진입");  
7     return "finally가 최종 가로챈 결과!"; // 가로챈 발생  
8   }  
9 }
```

return 덮어쓰기 결과 확인

```
1 console.log(checkReturnFlow());  
2 // 출력 결과:  
3 // "try 블록 진입"  
4 // "finally 블록 진입"  
5 // "finally가 최종 가로챈 결과!"
```

예외 처리와 블록 스코프

예외 처리와 블록 스코프 (Block Scope)

try, catch, finally 블록 스코프 내부의 const, let 변수가 외부 및 다른 블록에서 공유되지 않고 고립되는 스코프 구조

- **블록 레벨 변수:** try 블록 안에서 선언된 변수는 catch 나 finally 블록 및 하위 스코프 외부에서 접근할 수 없음
- **공통 변수 설계:** 여러 블록에서 공유하거나 예외 처리 후에도 활용하려면 상위 스코프에서 let 으로 먼저 공통 선언해야 함

예외 처리 스코프 안티패턴

```
1 function badScope() {  
2   try {  
3     const data = "민감한 데이터";  
4   } catch (err) {  
5     // console.log(data); // ReferenceError! (data 변수 참조 불가)  
6   }  
7 }
```

상위 스코프 공통 변수 설계

```
1  function goodScope() {  
2    let data = null; // 상위 스코프에 let 선언 및 초기화  
3  
4    try {  
5      data = "성공적으로 불러온 데이터";  
6    } catch (err) {  
7      console.error("오류 시점 데이터 참조:", data);  
8    }  
9  
10   console.log("최종 결과:", data); // 외부 스코프에서 활용 가능  
11 }  
12 goodScope();
```